



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
IEE2690: LABORATORIO DE SISTEMAS CIBERFÍSICOS

Proyecto 1

Laboratorio de Sistemas Ciberfísicos Grupo 1

Gabriel Andrés Aldunate Campos
Gabriel Armando Fuentes Guzmán
José Tomás Sandoval Flores
Michel Antonio Barrueto Gutiérrez

11 de abril de 2026

Índice

1. Primer Módulo	3
1.1. Conexión al PLC	3
1.1.1. Conexión entre PC y PLC	3
1.1.2. Carga del programa y monitoreo del estado del PLC	3
1.1.3. Descripción del proyecto y del programa de prueba básico	3
1.2. Conexión con la planta simulada mediante Modbus TCP	4
1.3. Lectura de sensores y escritura de actuadores	4
1.3.1. Monitoreo de variables	4
1.3.2. Evidencia del efecto de valores críticos en el comportamiento del proceso . . .	4
1.3.3. Descripción de las pruebas realizadas y comportamiento dinámico del proceso	5
1.4. Publicación de variables mediante OPC	5
1.4.1. Modelo de información del servidor OPC Mitsubishi	5
1.4.2. Configuración del servidor OPC	6
1.4.3. Address Space del servidor OPC	6
1.5. Interacción con cliente OPC	7
1.5.1. Conexión entre cliente OPC y servidor OPC	7
1.5.2. Método de seguridad mediante certificados en OPC UA	7
1.5.3. Variables en tiempo real y escritura desde cliente OPC	7
1.5.4. Descripción del rol de OPC UA en sistemas industriales	8
2. Segundo módulo	8
2.1. Planteamiento del problema de control	8
2.1.1. Objetivos del control y compensación de perturbaciones	8
2.1.2. Diagrama de control	8
2.2. Diseño e implementación del controlador	9
2.2.1. Ecuaciones y estructura del controlador utilizado	9
2.2.2. Implementación del controlador en el PLC	10
2.2.3. Objetivo del control y compensación de perturbaciones	10
2.2.4. Diagrama de flujo del proceso de control	10
2.2.5. Programa completo implementado en el PLC	11
2.3. Variables publicadas en el server OPC	12
2.4. Monitoreo y escritura en tiempo real a través del OPC UA	13
2.5. Desarrollo del Dashboard	13
2.5.1. Descripción de las herramientas utilizadas para implementar el dashboard . .	13
2.5.2. Diagrama de la arquitectura del dashboard y conexión del cliente OPC . . .	14
2.5.3. Implementación de la comunicación con el servidor OPC UA	14
2.5.4. Implementación de la pantalla del dashboard	15
2.6. Sintonización y evaluación de desempeño del controlador	15
2.6.1. Estrategia para la sintonización del controlador	15
3. Referencias	16
4. Anexo	17

1. Primer Módulo

1.1. Conexión al PLC

1.1.1. Conexión entre PC y PLC

Luego de realizar la configuración de las direcciones IP correspondientes, fue posible establecer conexión con el dispositivo, como se muestra en la Figura 1.

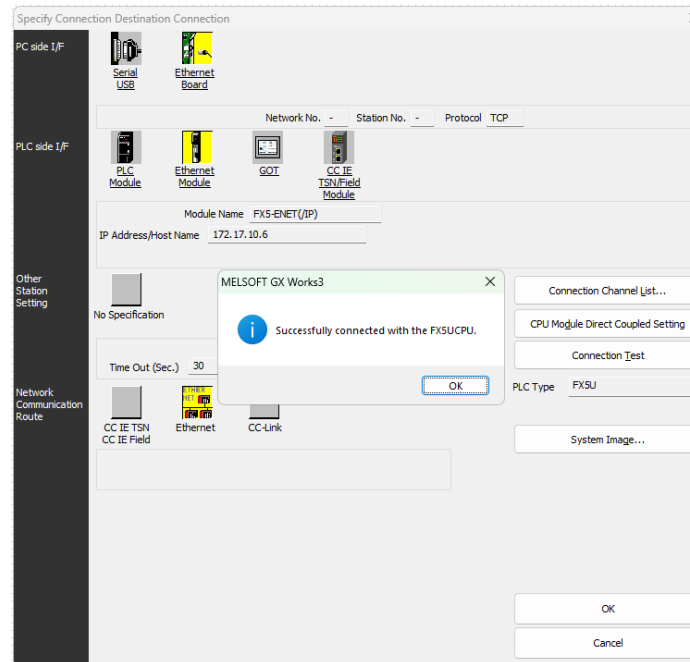


Figura 1: Conexión entre el PC y el PLC

1.1.2. Carga del programa y monitoreo del estado del PLC

Una vez establecida la conexión, se implementó un programa básico en lenguaje Ladder. En este, una señal de entrada $X0$ controla el estado de la salida $Y0$. Este ejemplo fue extraído del Manual de Iniciación al PLC (publicado en *Canvas*) y su estructura se presenta en la Figura 2.

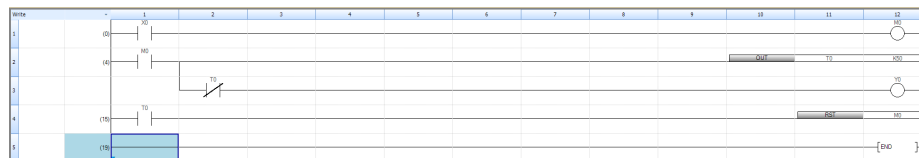


Figura 2: Programa en lenguaje Ladder

Tras cargar el programa en el PLC, se verificó el comportamiento esperado en la salida $Y0$. En la Figura 3 se observa cómo dicha salida se activa al modificar la entrada $X0$, confirmando así la correcta carga y ejecución del programa.

1.1.3. Descripción del proyecto y del programa de prueba básico

El programa de prueba corresponde al propuesto en el manual. Este consiste en que, al recibir una señal de activación en la entrada $X0$, la salida $Y0$ se enciende durante 5 segundos y luego se

Watch 1[Watching]						
ON OFF ON/OFF toggle Update						
Name	Current Value	Display Format	Data Type	English	Forced Input/Output Status	Device Test with Execution...
X0	ON	BIN	Bit		--	--
Y0	ON	BIN	Bit		--	--

Figura 3: Verificación del funcionamiento del programa

apaga, permaneciendo en ese estado hasta recibir una nueva activación.

1.2. Conexión con la planta simulada mediante Modbus TCP

En esta etapa se realizó la configuración del protocolo Modbus TCP/IP en el PLC. Esta configuración se muestra en la Figura 4.

Setting Item														
Latency Time 1 s (0s to 255s)														
Setting No.	Communication Pattern	Communication Setting (Execution Interval [ms])	Communication Destination (IP Address)		Target P.C. No.	Bit Device				Word Device				Communication Timeout Period [ms]
			Source	Destination		Points	Type	Start	End	Points	Type	Start	End	Monitoring Time at Error [s]
1	Read	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
2	Write	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
3	Read	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
4	Write	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
5	Read	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
6	Write	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
7	Read	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
8	Write	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
9	Read	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
10	Write	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
11	Read	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
12	Write	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
13	Read	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
14	Write	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
15	Read	Fixed Interval	100	MODBUS TCP172.17.10.10	Host Station(172.17.10.6)	Not Specified	->			0	D	0	1	1000
16							->							
17							->							
18							->							
19							->							
20							->							
21							->							
22							->							
23							->							
24							->							

Figura 4: Configuración de Modbus TCP

1.3. Lectura de sensores y escritura de actuadores

1.3.1. Monitoreo de variables

Luego de establecer la conexión con la planta simulada mediante Modbus TCP, se realizó la lectura de sensores y la escritura de actuadores con un valor fijo de 0.8. El monitoreo de estas variables se presenta en la Figura 5.

Name	Current Value	Display Format	Data Type	English	Forced Input/Output Status	Device Test with Execution...
nivel_c1	4.384490	--	FLOAT [Single Precision]		--	--
nivel_c2	4.375521	--	FLOAT [Single Precision]		--	--
nivel_c3	4.364325	--	FLOAT [Single Precision]		--	--
nivel_c4	4.350288	--	FLOAT [Single Precision]		--	--
nivel_c5	4.332641	--	FLOAT [Single Precision]		--	--
flujo_aire	0.000000	--	FLOAT [Single Precision]		--	--
ley_conc	1.376864	--	FLOAT [Single Precision]		--	--
recuperac...	99.998604	--	FLOAT [Single Precision]		--	--
ley_entrada	0.012558	--	FLOAT [Single Precision]		--	--
ley_cola	1.998298E-006	--	FLOAT [Single Precision]		--	--
D100	0.000000	--	FLOAT [Single Precision]		--	--
D0	4.384490	--	FLOAT [Single Precision]		--	--

Figura 5: Monitoreo de variables del sistema

1.3.2. Evidencia del efecto de valores críticos en el comportamiento del proceso

Inicialmente, se implementó un código encargado de leer las variables de interés y de escribir directamente un valor fijo de 0.8 en los actuadores, tal como se muestra en la Figura 6.

La aplicación de este valor constante da lugar a los niveles observados en la Figura 5. En la siguiente subsección se analiza cómo estos niveles varían en función de los flujos asignados.

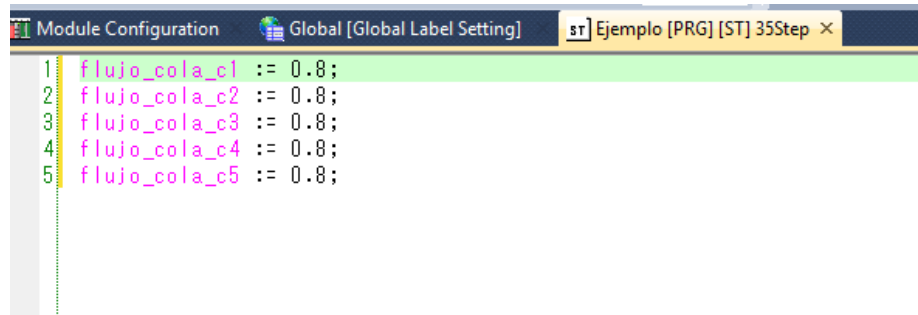


Figura 6: Escritura de un valor fijo en los actuadores

1.3.3. Descripción de las pruebas realizadas y comportamiento dinámico del proceso

Las primeras pruebas consistieron en la aplicación de valores fijos en los actuadores, lo que permitió observar la respuesta del sistema en condiciones simples. No obstante, las variaciones observadas durante la implementación de los controladores PID propuestos permiten caracterizar de mejor manera la dinámica del proceso.

En particular, el sistema de flotación presenta un alto grado de acoplamiento entre sus celdas, dado que la salida de cada una constituye la entrada de la siguiente. Esta característica se refleja en la respuesta global del sistema.

En la Figura 7 se observa que los flujos disminuyen de manera significativa una vez que el sistema alcanza el *setpoint*, en el contexto del experimento de control que se detallará en secciones posteriores. Este comportamiento evidencia la interacción entre las celdas y la influencia de la estrategia de control implementada.

Name	Current Value	Display For...	Data Ty
nivel_c1	4.350992	--	FLOAT
nivel_c2	4.083306	--	FLOAT
nivel_c3	3.578824	--	FLOAT
nivel_c4	4.400000	--	FLOAT
nivel_c5	3.487864	--	FLOAT
flujoCola_c1	0.000000	--	FLOAT
flujoCola_c2	6.160597E-005	--	FLOAT
flujoCola_c3	0.966753	--	FLOAT
flujoCola_c4	0.096675	--	FLOAT
flujoCola_c5	0.009668	--	FLOAT
flujoAire	0.700000	--	FLOAT
leyConc	19.620916	--	FLOAT
recuperacion	0.000000	--	FLOAT
leyEntrada	0.012558	--	FLOAT
leyCola	0.062459	--	FLOAT

Figura 7: Comportamiento dinámico del proceso

1.4. Publicación de variables mediante OPC

1.4.1. Modelo de información del servidor OPC Mitsubishi

El modelo de un servidor OPC está basado en el *Address Space*. Esta es una gran estructura que contiene todos los elementos del proyecto, llamados nodos. Esta estructura está subdividida en distintos tipos de referencias las cuales pueden ser jerárquicas (las cuales corresponden a carpetas con un orden jerárquico) o no jerárquicas (las cuales corresponden a contextos) por ejemplo, se podrían

tener varios sensores de nivel repartidos en varias carpetas una por cada celda pero referenciarlos a todos a un nodo maestro.

1.4.2. Configuración del servidor OPC

En esta etapa se configuró la dirección IP del módulo OPC, como se muestra en la Figura 8, logrando establecer su estado en línea (Figura 9). Esto permitió avanzar posteriormente en la conexión con clientes OPC.

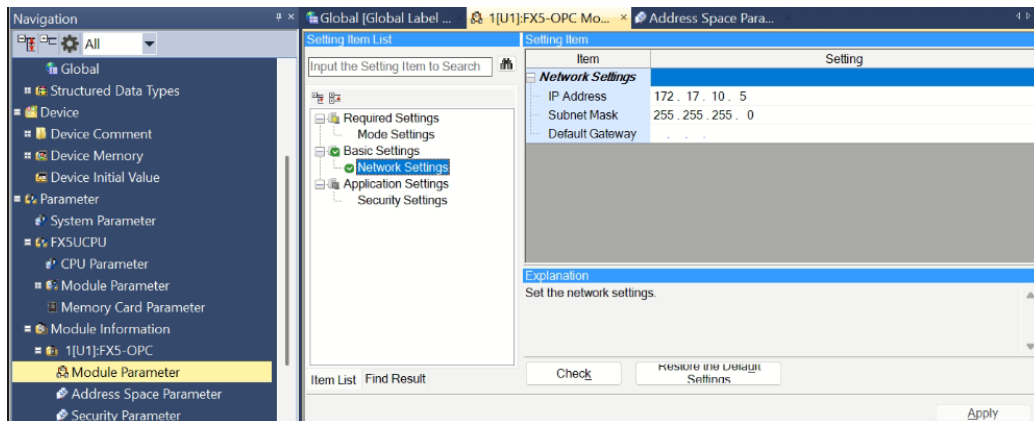


Figura 8: Configuración de IP del módulo OPC

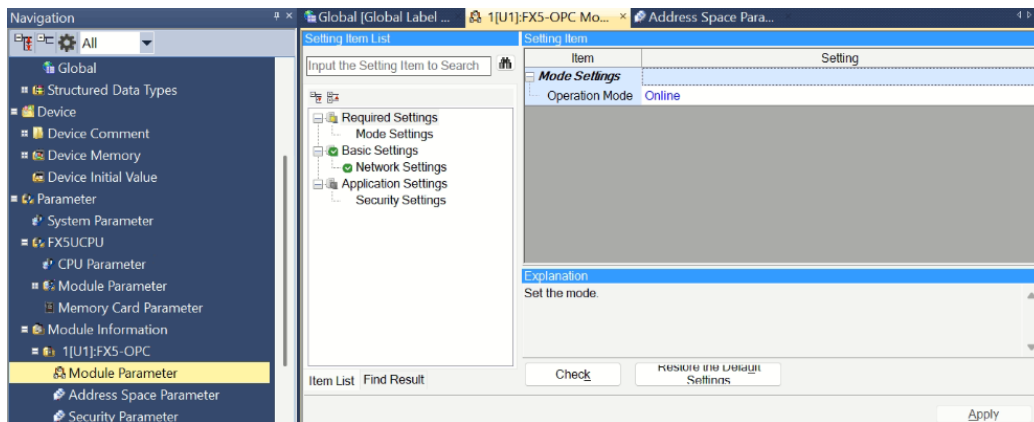


Figura 9: Estado en línea del servidor OPC

1.4.3. Address Space del servidor OPC

Dentro de la configuración del servidor OPC, se definieron las variables a exponer. Para ello, se creó un *Structured Data Type* denominado **sensor**, cuya instancia utilizada fue *sensor0*. Este tipo de dato agrupa los parámetros mostrados en la Figura 10.

En el *Address Space*, se habilitó el acceso de lectura y escritura para *sensor0* (Figura 11), incluyendo todos sus parámetros, cumpliendo así con los requerimientos solicitados.

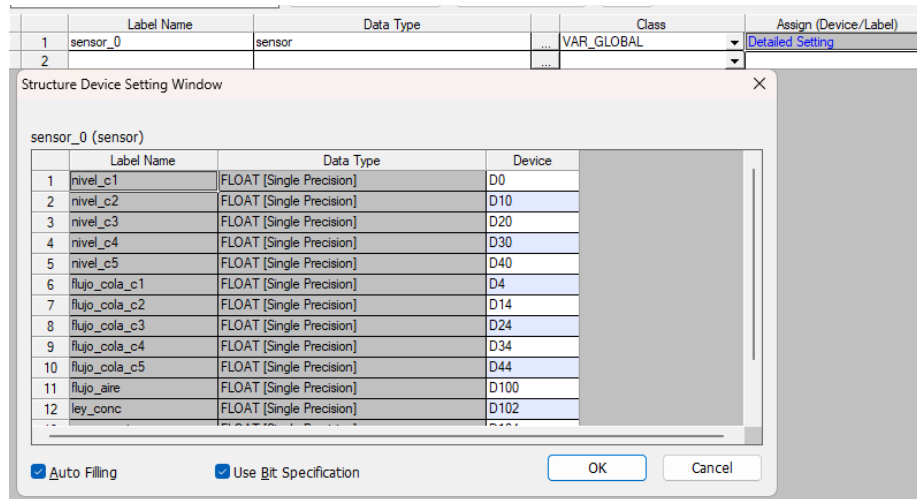


Figura 10: Parámetros de la instancia sensor0

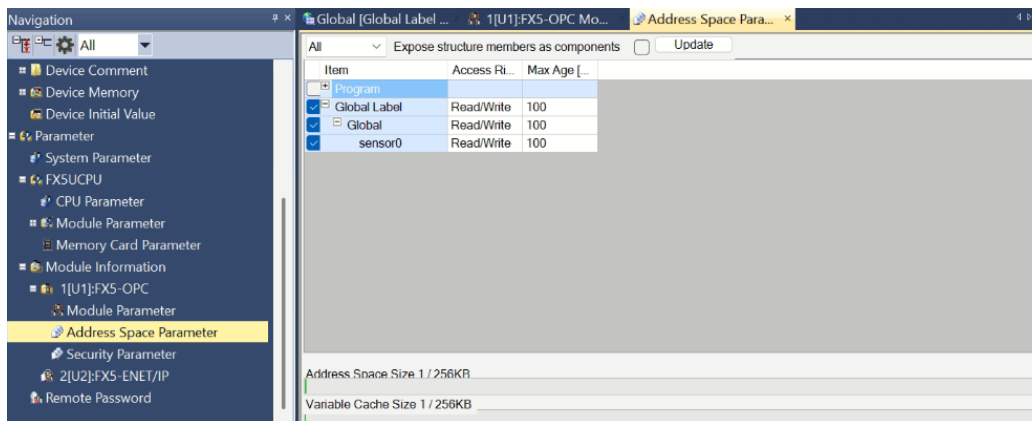


Figura 11: Address Space del servidor OPC

1.5. Interacción con cliente OPC

1.5.1. Conexión entre cliente OPC y servidor OPC

En la figura 12 es posible apreciar la conexión entre el cliente OPC y el servidor OPC que luego será utilizada para el manejo de variables.

1.5.2. Método de seguridad mediante certificados en OPC UA

Los certificados OPC UA son utilizados en la industria para identificar a servidores y clientes en una red OPC. Estos certificados incluyen la información del cliente, la entidad que entrega el certificado y una llave para verificar la "firma" de la llave privada asociada. Esto permite comunicaciones encriptadas en entornos industriales, lo que garantiza la seguridad al evitar que entidades no autorizadas se conecten a la red [1].

1.5.3. Variables en tiempo real y escritura desde cliente OPC

En la figura 12 es posible apreciar la lectura de los datos medidos en el sensor, además de tener la posibilidad de realizar la escritura de datos en los flujos de cola para modificar los niveles de las celdas.

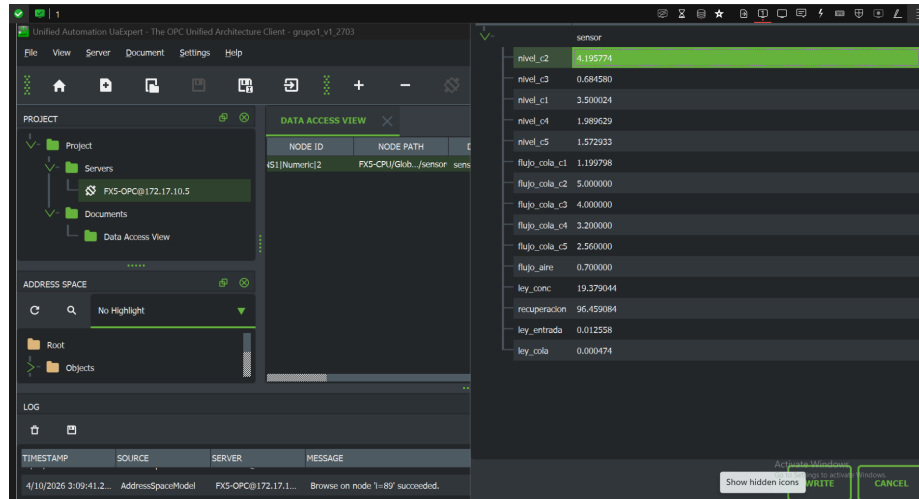


Figura 12: Lectura y manejo de variables desde UA EXPERT

1.5.4. Descripción del rol de OPC UA en sistemas industriales

A la hora de implementar sistemas de control industriales se necesita un protocolo para la comunicación entre dispositivos como lo son sensores, PLC y sistemas SCADA, este rol lo cumple el OPC UA el cual corresponde a un protocolo de comunicación entre máquinas (en el caso de este laboratorio sensores y plc), este permite una comunicación simple entre equipos de distintos proveedores, integrando además protocolos de seguridad, independencia de la plataforma (windows, linux, sistemas embebidos e infraestructura en la nube), todo esto en conjunto ha hecho que se vuelva un estándar en la industria siendo ampliamente utilizado por esta.

2. Segundo módulo

2.1. Planteamiento del problema de control

2.1.1. Objetivos del control y compensación de perturbaciones

El objetivo del sistema de control es lograr que cada celda mantenga el nivel de pulpa dentro de un rango previamente especificado.

La principal dificultad para cumplir este objetivo radica en el fuerte acoplamiento existente entre las celdas. En particular, cada celda recibe como entrada el flujo de salida de la celda anterior. Por ejemplo, la celda 2 recibe como entrada el flujo proveniente de la celda 1, lo que se traduce en una perturbación desde el punto de vista del controlador local. Esta perturbación no puede ser manipulada directamente, sino únicamente compensada a través de la acción de control.

Para mitigar este efecto, se propone la incorporación de una estrategia de *Feed Forward*. Esta consiste en utilizar la salida de cada celda como una señal adicional para el controlador de la celda siguiente, permitiendo anticipar el efecto de las perturbaciones y mejorar la respuesta del sistema.

2.1.2. Diagrama de control

A continuación, se muestra el diagrama de control correspondiente a una celda individual.

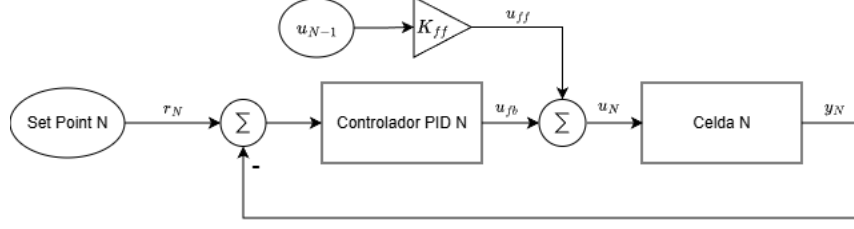


Figura 13: Diagrama de control para la celda N

La variable manipulada u_N se define como la suma de la acción de control generada por el PID y un término de *feedforward*, proporcional a la variable manipulada de la celda anterior:

$$u_N = u_{PID,N} + K_{ff} \cdot u_{N-1}$$

Esta formulación se adopta debido a que la dinámica de cada celda está directamente influenciada por el flujo de cola proveniente de la celda anterior. En consecuencia, si dicho flujo aumenta significativamente, la celda siguiente debe ajustar su acción de control en la misma dirección para mantener el nivel en torno a la referencia.

2.2. Diseño e implementación del controlador

2.2.1. Ecuaciones y estructura del controlador utilizado

- **Limitación del setpoint:** Para asegurar la estabilidad operativa, el setpoint ingresado se restringe a un rango de operación seguro (entre 2.0 y 5.0 metros):

$$SP_{\lim}[k] = \text{sat}(2,0, SP_{\text{raw}}[k], 5,0) \quad (1)$$

- **Cálculo del error:**

$$e[k] = SP_{\lim}[k] - PV[k] \quad (2)$$

- **Acción proporcional:**

$$P[k] = K_p \cdot e[k] \quad (3)$$

- **Acción integral (Aproximación de Tustin con Clamping):** Se utiliza integración trapezoidal para el cálculo del término integral. Para evitar el fenómeno de *windup*, se implementa una técnica de **clamping directo** sobre el acumulador $I[k]$, asegurando que el término integral no exceda los límites físicos del actuador:

$$I_{\text{raw}}[k] = I[k-1] + \frac{K_p}{T_i} \cdot \frac{T_s}{2} (e[k] + e[k-1]) \quad (4)$$

$$I[k] = \text{sat}(u_{\min}, I_{\text{raw}}[k], u_{\max}) \quad (5)$$

Esta modificación garantiza que el controlador responda instantáneamente cuando la variable de proceso cruza el setpoint, eliminando el retraso por desaturación.

- **Acción derivativa (Filtro Tustin sobre PV):** Se implementa la derivada sobre la variable de proceso para evitar el *derivative kick*. La ecuación en diferencias derivada de la transformación bilineal (Tustin) es:

$$D[k] = \frac{2T_d}{2T_d + T_s} D[k-1] - \frac{2K_p T_d}{2T_d + T_s} (PV[k] - PV[k-1]) \quad (6)$$

- **Señal de control total:** La suma de las tres acciones define la salida antes de la saturación final:

$$u_{\text{pre}}[k] = P[k] + I[k] + D[k] \quad (7)$$

- **Saturación de la salida y bandera de estado:** La salida final se limita al rango operacional $[0, 5]$. Se activa una señal booleana $bSaturated$ si el controlador alcanza sus límites:

$$u[k] = \text{sat}(u_{\min}, u_{\text{pre}}[k], u_{\max}) \quad (8)$$

$$bSaturated = \begin{cases} \text{True}, & \text{si } u_{\text{pre}}[k] \neq u[k] \\ \text{False}, & \text{en otro caso} \end{cases} \quad (9)$$

- **Actualización de memorias:** Al cierre de cada ciclo de escaneo ($T_s = 10ms$):

$$e[k-1] \leftarrow e[k] \quad (10)$$

$$PV[k-1] \leftarrow PV[k] \quad (11)$$

2.2.2. Implementación del controlador en el PLC

El código del controlador implementado en el PLC se encuentra disponible en el anexo. Para su diseño se utilizaron *Function Blocks*, los cuales emplean variables locales, tal como se muestra en la Figura 14.

Label Name	Data Type	Class
1 bManual	Bit	VAR_INPUT
2 bReset	Bit	VAR_INPUT
3 bSaturated	Bit	VAR_OUTPUT
4 fSetpoint	FLOAT (Single Precision)	VAR_INPUT
5 fPv	FLOAT (Single Precision)	VAR_INPUT
6 fKp	FLOAT (Single Precision)	VAR_INPUT
7 fTi	FLOAT (Single Precision)	VAR_INPUT
8 fTd	FLOAT (Single Precision)	VAR_INPUT
9 fTs	FLOAT (Single Precision)	VAR_INPUT
10 fLimMax	FLOAT (Single Precision)	VAR_INPUT
11 fLimMin	FLOAT (Single Precision)	VAR_INPUT
12 fManualOut	FLOAT (Single Precision)	VAR_INPUT
13 fOutput	FLOAT (Single Precision)	VAR_OUTPUT
14 fError	FLOAT (Single Precision)	VAR
15 fTerm	FLOAT (Single Precision)	VAR
16 fOutputSet	FLOAT (Single Precision)	VAR
17 fDTerm	FLOAT (Single Precision)	VAR
18 fPv_prev	FLOAT (Single Precision)	VAR
19 fError_prev	FLOAT (Single Precision)	VAR
20 fSP_limited	FLOAT (Single Precision)	VAR
21		
22		
23		
24		

Figura 14: Variables locales del *Function Block*

2.2.3. Objetivo del control y compensación de perturbaciones

Como se mencionó anteriormente, el objetivo del sistema de control es mantener los niveles de las celdas dentro del rango especificado.

En cuanto a la compensación de perturbaciones, se implementó una estrategia de *Feed Forward*, la cual se refleja en la siguiente línea de código:

```
sensor0.flujoCola_c2 := LIMIT(0.0, PID_2.fOutput + (fK_ff * sensor0.flujoCola_c1), 5.0);
```

Esta expresión permite ajustar la señal de salida incorporando el efecto de la celda anterior, lo que contribuye a anticipar perturbaciones y mejorar el desempeño del sistema.

2.2.4. Diagrama de flujo del proceso de control

Este diagrama se muestra en la figura 15

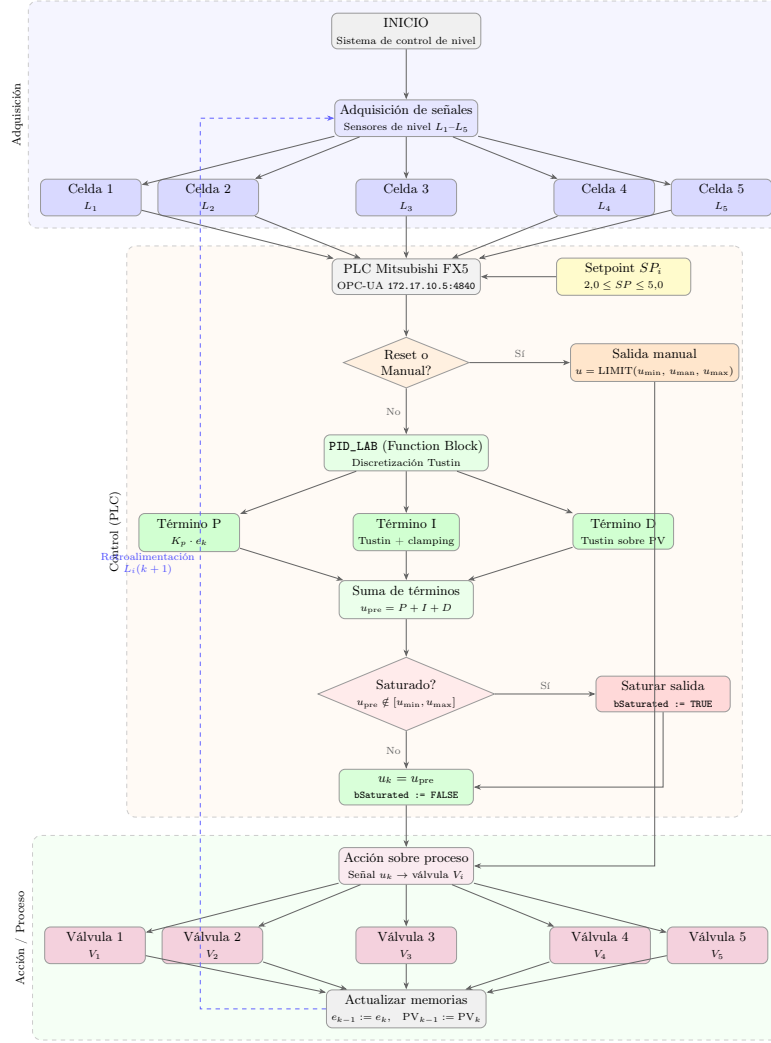


Figura 15: Diagrama de flujo: adquisición, control PID Tustin y acción sobre 5 válvulas de flotación.

2.2.5. Programa completo implementado en el PLC

El programa completo se incluye en el anexo. En términos generales, la implementación consta de cinco instancias de un controlador PID genérico.

Este controlador fue desarrollado como un *Function Block*, el cual es llamado desde el programa principal para generar las cinco instancias. Cada una de estas instancias opera con parámetros y *setpoints* específicos, adaptados a las condiciones particulares de cada celda.

A continuación se muestran fotos del programa en el PLC.

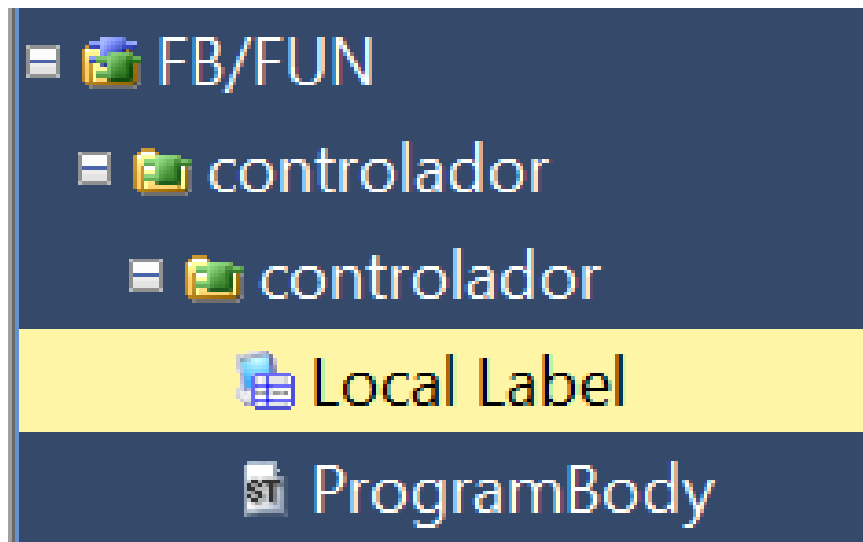


Figura 16: Programa del function block

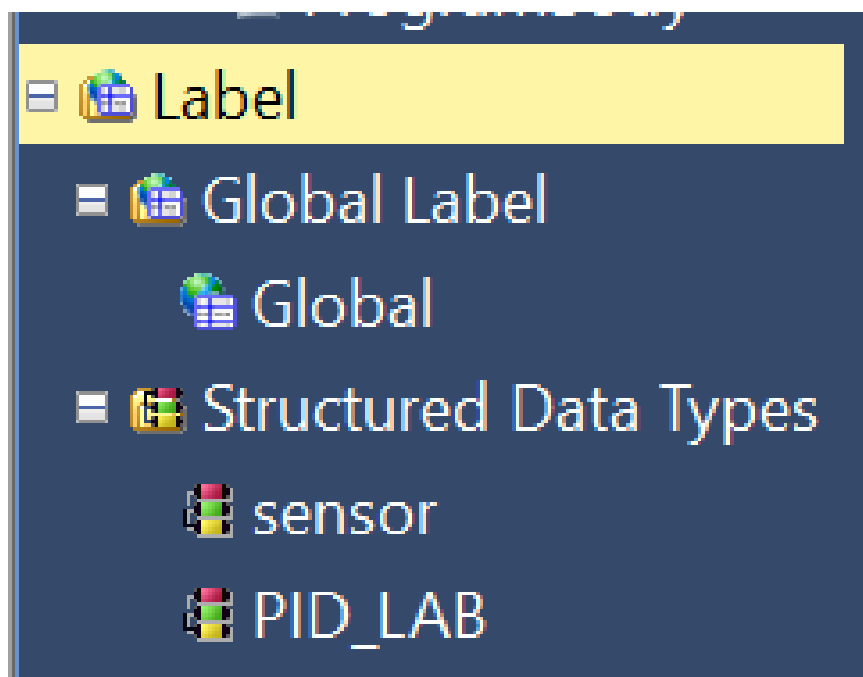


Figura 17: Programa principal

2.3. Variables publicadas en el server OPC

Los siguientes datos del sensor son publicados en el servidor:

Sensor de proceso (sensor0)

- Nivel de pulpa, celdas 1-5 [m]
- Flujo de cola, celdas 1-5 [m³/s]

- Flujo de aire [m^3/s]
- Ley del concentrado [%]
- Recuperación metalúrgica [%]
- Ley de entrada [%]
- Ley de cola [%]

Controladores PID (PID_1 a PID_5)

Cada controlador publica los siguientes campos:

- **Parámetros (escritura desde Python):** setpoint, K_p , T_i , T_d , PV y salida del controlador.

Variables globales

- K_p base
- T_i base
- T_d base
- Ganancia de *feedforward* K_{ff}

2.4. Monitoreo y escritura en tiempo real a través del OPC UA

Como se menciona en la sección 1.5.3. Se logró monitorear y escribir variables desde el servidor OPC, lamentablemente se perdió la conexión con el PLC antes de demostrar su funcionamiento con el controlador diseñado y simplemente fue posible registrar mediante fotos el manejo del sensor.

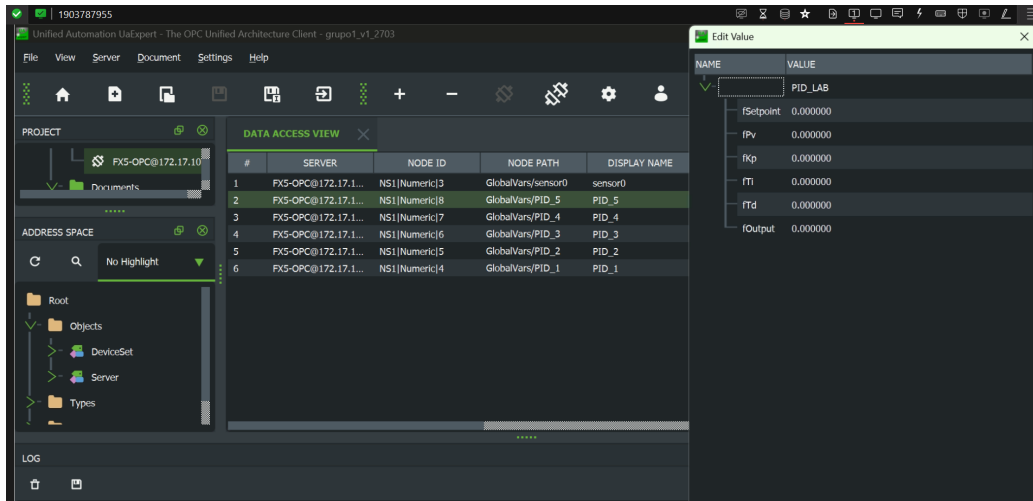


Figura 18: Conexión con el módulo OPC

2.5. Desarrollo del Dashboard

2.5.1. Descripción de las herramientas utilizadas para implementar el dashboard

El dashboard fue implementado con **Streamlit**, que es un framework de Python orientado al desarrollo rápido de aplicaciones web de datos y el cual fue recomendado dentro de las indicaciones propuestas. La comunicación con el PLC se realizó a través de la librería **asyncua**, el cual actúa como cliente OPC UA asíncrono para Python.

2.5.2. Diagrama de la arquitectura del dashboard y conexión del cliente OPC

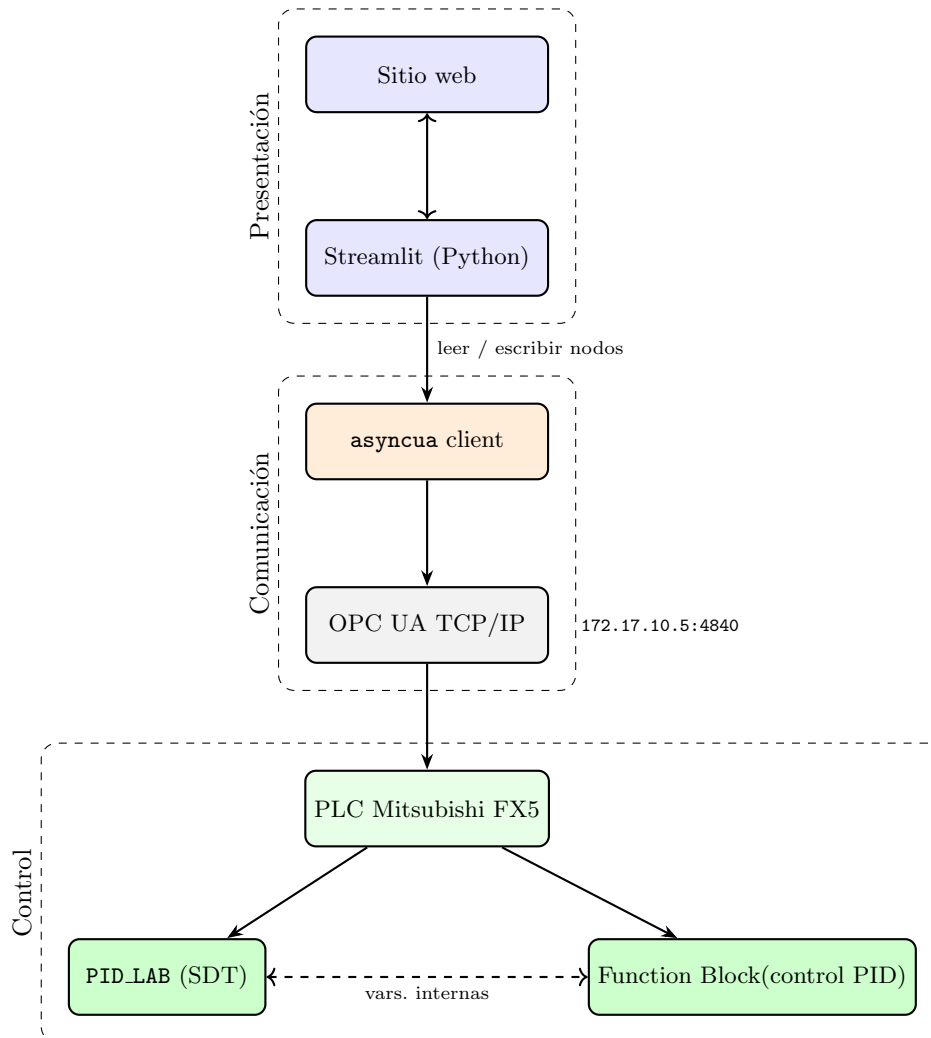


Figura 19: Arquitectura del dashboard y conexión al servidor OPC UA.

2.5.3. Implementación de la comunicación con el servidor OPC UA

La conexión al servidor OPC UA se habilitó en modo *inseguro* (sin cifrado ni autenticación) desde *Gx Works 3* para facilitar la conexión (en el contexto de industrias es preferible trabajar con certificados como se hizo en *UA expert*). En el lado cliente, la función `leer_nodos_bulk` establece una sesión temporal con el servidor, solicita múltiples nodos en una sola transacción y cierra la conexión al finalizar. Eso busca reducir la latencia acumulada frente a lecturas individuales.

```
1 async def leer_nodos_bulk(node_ids):
2     async with Client(url=OPC_URL) as client:
3         nodes = [client.get_node(nid) for nid in node_ids]
4         return await client.read_values(nodes)
```

Listing 1: Lectura masiva de nodos OPC UA.

Los nodos de sensores se identifican con NodeIds del espacio de nombres 1 (por ejemplo `ns=1;i=16` para nivel de celda 1). Los parámetros de los controladores PID se organizan en bloques consecutivos de seis nodos a partir de `ns=1;i=29`, desplazados según el índice de celda. En esta estructura

se puede apreciar el *Structured Data Type* PID_LAB definido en el PLC, el cual agrupa las variables de comunicación de cada controlador (setpoint, variable de proceso, ganancias K_p , T_i , T_d y salida). Por otra parte, la lógica de control está en un *function block* interno del PLC; el SDT funciona únicamente como interfaz de lectura y escritura.

La escritura de parámetros se hace mediante la función `escribir_nodo`, que construye un `DataValue` tipo `Float` y lo envía al nodo correspondiente:

```
1 async def escribir_nodo(node_id, valor, tipo=ua.VariantType.Float):
2     async with Client(url=OPC_URL) as client:
3         node = client.get_node(node_id)
4         await node.write_value(
5             ua.DataValue(ua.Variant(float(valor), tipo))
6         )
```

Listing 2: Escritura de un parámetro PID en el PLC.

2.5.4. Implementación de la pantalla del dashboard

En la figura 20 es posible apreciar que el dashboard implementado permite monitorear los datos de cada celda a la vez que se construye un gráfico para facilitar la visualización.

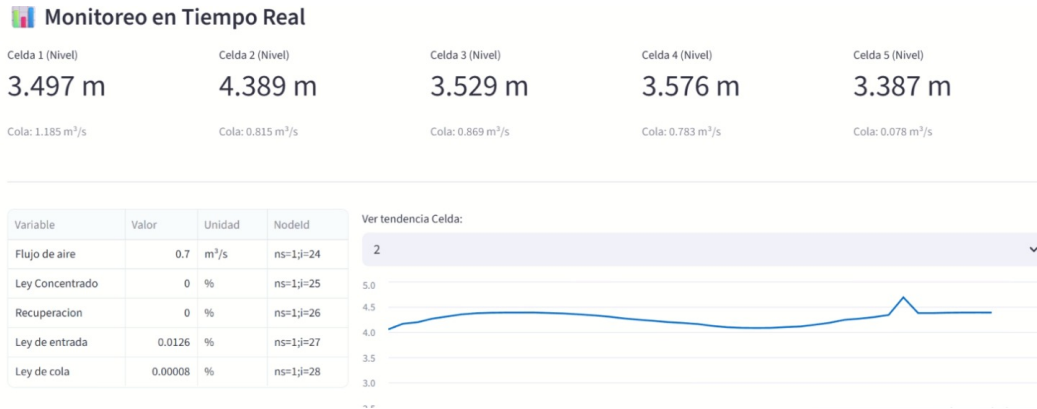


Figura 20: Implementación del dashboard

2.6. Sintonización y evaluación de desempeño del controlador

Debido a la desconexión del PLC y el módulo ethernet no se pudieron modificar los parámetros del PID discreto programado para un funcionamiento óptimo.

2.6.1. Estrategia para la sintonización del controlador

La estrategia que se hubiera seguido hubiera sido la siguiente.

- Aumentar K_p hasta alcanzar un tiempo de respuesta razonable
- Aumentar T_i para eliminar error en estado estacionario
- Si el flujo de cola de la celda anterior es muy alto aumentar K_{ff} para que influya más en la variable manipulada.

3. Referencias

- [1] T. Tosun. «OPC UA certificate explained. »dirección: <https://www.ptc.com/en/blogs/iiot/opc-ua-certificate-explained>.

4. Anexo

Cápsula

La cápsula se encuentra en el siguiente link

```
1 // 1. Reset y Manual
2 IF bReset THEN
3     fITerm      := 0.0;
4     fOutput     := 0.0;
5     fError_prev := 0.0;
6     fPV_prev    := fPV;
7     bSaturated  := FALSE;
8     RETURN;
9 END_IF;
10
11 IF bManual THEN
12     fOutput     := LIMIT(fLimMin, fManualOut, fLimMax);
13     fITerm      := fOutput;
14     fPV_prev    := fPV;
15     fError_prev := fSetpoint - fPV;
16     RETURN;
17 END_IF;
18
19 // 2. Init de memorias
20 IF NOT binitdone THEN
21     fPV_prev    := fPV;
22     fError_prev := fSetpoint - fPV;
23     fITerm      := 0.0;
24     binitdone   := TRUE;
25 END_IF;
26
27 // 3. Preparacion
28 fSP_limited := LIMIT(2.0, fSetpoint, 5.00);
29 fError      := fSP_limited - fPV;
30
31 // 4. Integral Tustin con anti-windup clasico (clamping)
32 fITerm_delta := (fKp / fTi) * (fTs / 2.0) * (fError + fError_prev);
33
34 IF (fTi > 0.0) AND (fTs > 0.0) THEN
35     // Sumar el delta al ITerm
36     fITerm := fITerm + fITerm_delta;
37
38     // CLAMPING: Evitar que el integrador acumule mas alla de los limites
39     // fisicos
40     IF fITerm > fLimMax THEN
41         fITerm := fLimMax;
42     ELSIF fITerm < fLimMin THEN
43         fITerm := fLimMin;
44     END_IF;
45 END_IF;
46
47 // 5. Derivativa Tustin sobre PV
48 IF (fTd > 0.0) AND (fTs > 0.0) THEN
49     fDTerm := (2.0 * fTd / (2.0 * fTd + fTs)) * fDTerm
50     - (2.0 * fKp * fTd / (2.0 * fTd + fTs)) * (fPV - fPV_prev);
51 ELSE
52     fDTerm := 0.0;
53 END_IF;
```

```

53
54 // 6. Salida total
55 fOutPreSat := (fKp * fError) + fITerm + fDTerm;
56
57 // 7. Saturacion
58 IF fOutPreSat > fLimMax THEN
59     fOutput      := fLimMax;
60     bSaturated := TRUE;
61     ELSIF fOutPreSat < fLimMin THEN
62         fOutput      := fLimMin;
63         bSaturated := TRUE;
64     ELSE
65         fOutput      := fOutPreSat;
66         bSaturated := FALSE;
67 END_IF;
68
69 // 8. Memorias
70 fPV_prev      := fPV;
71 fError_prev   := fError;

```

Listing 3: Implementación de controlador PID en PLC (Structured Text)

```

1 // INIT:
2 IF NOT bInit THEN
3     fKp_Base := -1.2;
4     fTi_Base := 20.0;
5     fTd_Base := 0.0;
6     fK_ff    := 0.1;
7
8     // PID 1
9     PID_1.fSetpoint := 4.350992;
10    PID_1.fKp := fKp_Base;
11    PID_1.fTi := fTi_Base;
12    PID_1.fTd := fTd_Base;
13
14    // PID 2
15    PID_2.fSetpoint := 4.083306;
16    PID_2.fKp := fKp_Base;
17    PID_2.fTi := fTi_Base;
18    PID_2.fTd := fTd_Base;
19
20    // PID 3
21    PID_3.fSetpoint := 3.5;
22    PID_3.fKp := fKp_Base;
23    PID_3.fTi := fTi_Base;
24    PID_3.fTd := fTd_Base;
25
26    // PID 4
27    PID_4.fSetpoint := 4.4;
28    PID_4.fKp := fKp_Base;
29    PID_4.fTi := fTi_Base;
30    PID_4.fTd := fTd_Base;
31
32    // PID 5
33    PID_5.fSetpoint := 3.5;
34    PID_5.fKp := fKp_Base;
35    PID_5.fTi := fTi_Base;
36    PID_5.fTd := fTd_Base;
37

```

```

38     bInit := TRUE;
39 END_IF;
40
41
42 // PV
43 PID_1.fPv := sensor0.nivel_c1;
44 PID_2.fPv := sensor0.nivel_c2;
45 PID_3.fPv := sensor0.nivel_c3;
46 PID_4.fPv := sensor0.nivel_c4;
47 PID_5.fPv := sensor0.nivel_c5;
48
49
50 // EJECUCI N FB
51 ctrl_1(
52   fSetpoint := PID_1.fSetpoint,
53   fPv       := PID_1.fPv,
54   fKp       := PID_1.fKp,
55   fTi       := PID_1.fTi,
56   fTd       := PID_1.fTd,
57   fTs       := 0.01, // 10 ms forzados (en segundos)
58   fLimMax   := 5.0,   // L mite m ximo forzado
59   fLimMin   := 0.0,   // L mite m nimo forzado
60   bManual   := FALSE,
61   fManualOut := 0.0,
62   bReset    := FALSE
63 );
64 PID_1.fOutput := ctrl_1.fOutput;
65
66
67 ctrl_2(
68   fSetpoint := PID_2.fSetpoint,
69   fPv       := PID_2.fPv,
70   fKp       := PID_2.fKp,
71   fTi       := PID_2.fTi,
72   fTd       := PID_2.fTd,
73   fTs       := 0.01,
74   fLimMax   := 5.0,
75   fLimMin   := 0.0,
76   bManual   := FALSE,
77   fManualOut := 0.0,
78   bReset    := FALSE
79 );
80 PID_2.fOutput := ctrl_2.fOutput;
81
82
83 ctrl_3(
84   fSetpoint := PID_3.fSetpoint,
85   fPv       := PID_3.fPv,
86   fKp       := PID_3.fKp,
87   fTi       := PID_3.fTi,
88   fTd       := PID_3.fTd,
89   fTs       := 0.01,
90   fLimMax   := 5.0,
91   fLimMin   := 0.0,
92   bManual   := FALSE,
93   fManualOut := 0.0,
94   bReset    := FALSE
95 );
96 PID_3.fOutput := ctrl_3.fOutput;

```

```

97
98
99 ctrl_4(
100 fSetpoint := PID_4.fSetpoint,
101 fPv       := PID_4.fPv,
102 fKp       := PID_4.fKp,
103 fTi       := PID_4.fTi,
104 fTd       := PID_4.fTd,
105 fTs       := 0.01,
106 fLimMax   := 5.0,
107 fLimMin   := 0.0,
108 bManual   := FALSE,
109 fManualOut := 0.0,
110 bReset    := FALSE
111 );
112 PID_4.fOutput := ctrl_4.fOutput;
113
114
115 ctrl_5(
116 fSetpoint := PID_5.fSetpoint,
117 fPv       := PID_5.fPv,
118 fKp       := PID_5.fKp,
119 fTi       := PID_5.fTi,
120 fTd       := PID_5.fTd,
121 fTs       := 0.01,
122 fLimMax   := 5.0,
123 fLimMin   := 0.0,
124 bManual   := FALSE,
125 fManualOut := 0.0,
126 bReset    := FALSE
127 );
128 PID_5.fOutput := ctrl_5.fOutput;
129
130
131 //SALIDAS
132 sensor0.flujo_cola_c1 := PID_1.fOutput;
133 sensor0.flujo_cola_c2 := PID_2.fOutput + fK_ff * sensor0.flujo_cola_c1;
134 sensor0.flujo_cola_c3 := PID_3.fOutput + fK_ff * sensor0.flujo_cola_c2;
135 sensor0.flujo_cola_c4 := PID_4.fOutput + fK_ff * sensor0.flujo_cola_c3;
136 sensor0.flujo_cola_c5 := PID_5.fOutput + fK_ff * sensor0.flujo_cola_c4;

```

Listing 4: Control en cascada con compensacion feedforward entre celdas